

Entregable 3: Memoria final

Aarón Blanco Álvarez

Máster en ingeniería informática

**Universidad de Granada
Facultad de Ingeniería Informática**



**UNIVERSIDAD
DE GRANADA**

Memoria final – MiauMarket

Documentación del Modelo de Datos - MiauMarket

1. Introducción

La interfaz del repository define como se hace el acceso a datos, y la implementación conecta con los datos: backend con Retrofit y sesión local con DataStore.

En auth, el repository guarda y borra el token JWT. En products, el repository consulta el catálogo, aplica paginación y obtiene el detalle de cada producto.

La UI nunca accede directamente a Retrofit ni a DataStore; siempre pasa por el repository.

2. Persistencia local en Android (DataStore)

Nombre: *session_prefs*

Representa la sesión activa en el dispositivo.

Campo	Tipo Kotlin	Obligatorio	Descripción
jwt_token	String	No	JWT para enviar en Authorization: Bearer <token>.

Ejemplo real:

JSON

```
{
  "jwt_token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxliZW1haWwiOiJwZXBIQGV4YW1wbG
  UuY29tliwcm9sZSI6InVzZXlifQ.signature"
}
```

3. APIs que consume Android

Las consultas a el backend cuya ruta es */api*.

3.1. Auth

POST */api/auth/register*

Escritura de usuario y apertura de sesión.

Request:

Campo	Obligatorio	Tipo Kotlin
firstName	Sí	String
lastName	Sí	String
birthDate	Sí	String (yyyy-MM-dd)
email	Sí	String
password	Sí	String
role	No (default user)	String

Response (201):

JSON

```
{
  "token": "<jwt>",
  "user": {
    "id": 1,
    "firstName": "Pepe",
    "lastName": "Garcia",
    "birthDate": "1998-05-21T00:00:00.000Z",
    "email": "pepe@example.com",
    "role": "user"
  }
}
```

POST /api/auth/login

Inicio de sesión.

Request:

Campo	Obligatorio	Tipo Kotlin
email	Sí	String
password	Sí	String

Response (200):

JSON

```
{
  "token": "<jwt>",
  "user": {
    "id": 1,
    "firstName": "Pepe",
    "lastName": "Garcia",
    "birthDate": "1998-05-21T00:00:00.000Z",
    "email": "pepe@example.com",
    "role": "user"
  }
}
```

GET /api/auth/me

Lectura del usuario autenticado.

Header recomendado en Android: Authorization: Bearer <jwt_token>

Response (200):

JSON

```
{
  "user": {
    "id": 1,
    "firstName": "Pepe",
    "lastName": "Garcia",
    "birthDate": "1998-05-21T00:00:00.000Z",
    "email": "pepe@example.com",
    "role": "user"
  }
}
```

POST /api/auth/logout

Cierre de sesión.

Response (200):

JSON

```
{ "ok": true }
```

3.2. Products

GET */api/products*

Catálogo con la parte de páginas.

Request:

Campo	Tipo	Obligatorio
search	String	No
take	Int	No
skip	Int	No

Response (200):

JSON

```
{
  "total": 128,
  "take": 20,
  "skip": 0,
  "items": [
    {
      "id": 57,
      "source": "kiwoko",
      "title": "Rascador arbol para gatos",
      "price": 45.95,
      "rawPrice": "45.95 EUR",
      "currency": "EUR",
      "url": "https://www.kiwoko.com/gatos/rascador-x",
      "image": "https://cdn.kiwoko.com/rascador-x.jpg",
      "scrapedAt": "2026-04-07T10:32:12.000Z",
      "createdAt": "2026-04-07T10:33:01.000Z",
      "updatedAt": "2026-04-07T10:33:01.000Z"
    }
  ]
}
```

Otros Endpoints de Operaciones

- **GET** */api/products/:id*: Lectura de detalle por id numérico.
- **POST** */api/products/:id*: Creación manual de producto (requiere rol admin).
- **PUT** */api/products/:id*: Actualización de producto (requiere rol admin).
- **DELETE** */api/products/:id*: Borrado de producto (requiere rol admin).

4. DTOs Kotlin

ProductResponse

Campo	Tipo Kotlin	Obligatorio
id	Long	Sí
source	String	Sí
title	String	Sí
price	Double?	No
rawPrice	String?	No
currency	String	Sí
url	String	Sí
image	String?	No

scrapedAt	String	Sí
createdAt	String	Sí
updatedAt	String	Sí

ProductsListResponse

Campo	Tipo Kotlin	Obligatorio
total	Int	Sí
take	Int	Sí
skip	Int	Sí
items	List<ProductResponse>	Sí

UserResponse

Campo	Tipo Kotlin	Obligatorio
id	Long	Sí
firstName	String	Sí
lastName	String	Sí
birthDate	String	Sí
email	String	Sí
role	String	Sí

AuthResponse

Campo	Tipo Kotlin	Obligatorio
token	String	Sí
user	UserResponse	Sí

4. Relaciones y cardinalidad relevantes para Android

La app no usa Room ni una base de datos relacional local, sino que consume un backend REST y guarda únicamente la sesión en DataStore. Por eso, las relaciones entre datos son lógicas y vienen definidas por la API:

- **Usuario: Sesión en dispositivo (1 : 0..1):** Un usuario puede tener como máximo un *jwt_token* guardado en el dispositivo. Esa sesión se persiste en DataStore dentro de *session_prefs*.
- **Products (lista): Product (detalle) (1 : N):** El endpoint */api/products* devuelve varios productos dentro de *items*, y cada producto puede consultarse individualmente mediante */api/products/:id*.
- **Producto: Source (N : 1 lógica):** Cada producto incluye un campo *source* que indica su origen. No existe una tabla aparte para las fuentes(source), pero varios productos pueden compartir el mismo valor.

5. Decisiones de dataset que afectan a la app

- Se conservan *id*, *title*, *price*, *currency*, *image*, *url* y *source* porque son los campos usados por listado, detalle y filtros.
- Se descartan descripciones largas y metadatos no usados para reducir tráfico y complejidad en Android.
- **birthDate** se captura en la interfaz como **dd/MM/aaaa** y antes de enviar se transforma a **yyyy-MM-dd**, que es el formato esperado por la API.
- Los identificadores (id) se conservan como numéricos porque el backend los devuelve como enteros y eso facilita la navegación y la lectura de detalle.

Documentación de la Navegación - MiauMarket

6. Introducción de la Navegación

La idea del documento es detallar la fase de Navegación y Flujo de Pantallas de la aplicación MiauMarket, desarrollada como parte de las prácticas de la asignatura. Tras haber definido en el entregable anterior la estructura de datos y la integración con el backend, este segundo hito se centra en la experiencia interactiva del usuario y la arquitectura de navegación de la app.

El objetivo principal es establecer una estructura clara de pantallas y sus relaciones, garantizando un flujo de usuario coherente y funcional. Para ello, se ha implementado un sistema de navegación basado en Jetpack Compose, que permite el paso de información entre pantallas.

En las siguientes secciones se documenta cada una de las pantallas que componen el flujo principal que van desde la autenticación hasta la gestión de productos, con las rutas, argumentos y las acciones que permiten realizar al usuario.

7. Estructura de Pantallas

7.1. LoginScreen (Inicio de Sesión)

- **Representa:** formulario de autenticación.
- **Información que muestra:** email, contraseña y mensaje de error en caso de fallo.
- **Acciones:** iniciar sesión o navegar a registro.
- **Ruta:** LoginRoute.
- **Argumentos:** ninguno.
- **Relaciones:**
 - **Desde:** CatalogRoute cuando el usuario pulsa el acceso a sesión si no está registrado o RegisterRoute.
 - **Hacia:** CatalogRoute tras login exitoso, o RegisterRoute.

7.2. RegisterScreen (Registro)

- **Representa:** alta de usuario.
- **Información que muestra:** firstName, lastName, birthDate (selector de fecha), email, password.
- **Acciones:** registrar usuario o volver a login.
- **Ruta:** RegisterRoute.
- **Argumentos:** ninguno.
- **Relaciones:**
 - **Desde:** LoginRoute.
 - **Hacia:** CatalogRoute tras registro exitoso, o LoginRoute.

7.3. CatalogScreen (Catálogo Principal)

- **Representa:** pantalla principal de productos.
- **Información que muestra:** listado paginado, buscador, estado de carga, errores, acceso al carrito y menú de sesión.
- **Acciones:** buscar, cargar más al hacer scroll, abrir detalle, añadir al carrito, abrir carrito, logout; crear producto si el usuario es admin.
- **Ruta:** CatalogRoute.
- **Argumentos:** ninguno.
- **Relaciones:**
 - **Desde:** inicio de app LoginRoute, RegisterRoute, CartRoute (al volver atrás).
 - **Hacia:** ProductDetailRoute(id), ProductFormRoute(productId = null) (admin), CartRoute, LoginRoute.

7.4. ProductDetailScreen (Detalle del Producto)

- **Representa:** detalle de un producto concreto.
- **Información que muestra:** imagen, nombre, precio, fuente y enlace a la URL original.
- **Acciones:** volver atrás, añadir al carrito, abrir URL en navegador; editar/eliminar si admin.
- **Ruta:** ProductDetailRoute(id: Long).
- **Argumentos:**
 - id (**Long**, obligatorio): ID numérico del producto.
- **Relaciones:**
 - **Desde:** CatalogRoute.
 - **Hacia:** Volver atrás.

7.5. CartScreen (Carrito de Compras)

- **Representa:** resumen de compra en estado local de app.
- **Información que muestra:** productos añadidos, cantidad y total.
- **Acciones:** eliminar producto, vaciar carrito y botón de checkout.
- **Ruta:** CartRoute.
- **Argumentos:** ninguno.
- **Relaciones:**
 - **Desde:** CatalogRoute.
 - **Hacia:** Pantalla anterior al volver atrás.

7.6. ProductFormScreen (Gestión de Producto - Admin)

- **Representa:** creación/edición manual de producto.
- **Información que muestra:** campos de título, precio, imagen, URL, moneda y fuente.
- **Acciones:** guardar (create/update) y volver atrás.

- **Ruta:** ProductFormRoute(productId: Long? = null).
- **Argumentos:**
 - productId (**Long?**, opcional): null crea, valor numérico edita.
- **Relaciones:**
 - **Desde:** CatalogRoute (crear), ProductDetailRoute (editar).
 - **Hacia:** Vuelta atrás tras guardar o cancelar.

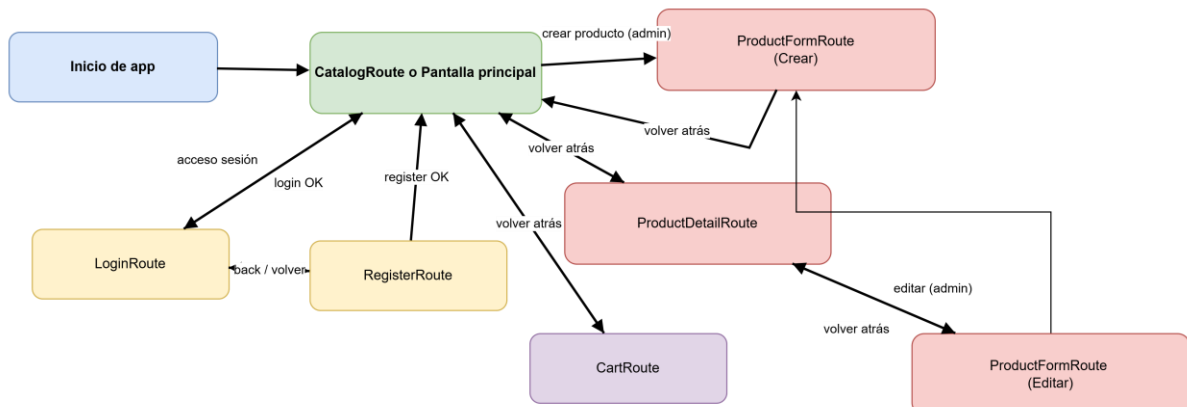
8. Recorrido Completo del Usuario (Caso de Uso Principal)

Caso de Uso: El usuario busca algo interesante para su gato al arrancar la app.

1. La app arranca en CatalogRoute.
2. El usuario busca productos y hace scroll para cargar más resultados.
3. Pulsa sobre un producto y entra en el detalle, ProductDetailRoute.
4. En el detalle, añade el producto al carrito (también por fuera).
5. Vuelve al catálogo y abre CartRoute desde el icono superior.
6. En carrito, puede eliminar líneas (compra) o vaciarlo completo.

9. Diagrama de navegación

He realizado un pequeño diagrama claro de navegación que muestra el flujo de navegación por encima de la aplicación, hecho con Draw.io



10. Justificación Técnica de las decisiones tomadas

Arquitectura (MVVM + Clean Architecture): He separado la aplicación en capas para que el código esté organizado y sea fácil de mantener. La pantalla (UI) no gestiona datos,

simplemente "observa" los cambios y se actualiza automáticamente cuando la información se recibe del backend.

Actividad Única (Single Activity Pattern): Toda la aplicación funciona dentro de una sola ventana (MainActivity). Esto hace que la navegación entre pantallas sea mucho más ligera, rápida y eficiente en el consumo de recursos.

Control del Ciclo de Vida: Utilizo Corrutinas para las tareas en segundo plano. Esto garantiza que, si el usuario cierra una pantalla, las descargas de datos pendientes se detengan solas, evitando que la app consuma memoria de forma innecesaria o falle. Un ejemplo son las cargas del catálogo de productos que van muy bien con eso.

Diseño Adaptable (Responsive): He diseñado las listas de productos para que se ajusten solas al tamaño de cualquier dispositivo, ya sea un móvil pequeño o una tablet, aprovechando siempre el espacio de la mejor manera. Ejemplo de ello, en el catálogo el uso de *GridCells.Adaptive*, en parte, ya usado para el mismo catálogo para la aplicación de WearOs.

Notificaciones: Se implementó un sistema de Notificaciones del sistema para informar de al usuario sobre acciones relevantes dentro de la aplicación, básicamente la confirmación al realizar un pedido.

Optimización UI/UX: Se incluyó un GIF animado en las pantallas de añadir productos, es totalmente innecesario, pero a la vez necesario como estrategia de marketing para hacer feliz a la gente al ver una pazuña de gatito. Esta mejora optimiza la experiencia visual (UX) y el feedback, reduciendo la sensación de tiempo de espera del usuario, aunque precisamente la esté alargando, pero esa es la idea.

DatStore: Se decidió guardar el token localmente para optimizar el tráfico de red y la experiencia de usuario. Carecía de sentido técnico solicitar el token a la API en cada inicio de aplicación o petición. Almacenarlo en DataStore permite que la sesión sea persistente entre reinicios del dispositivo y que las peticiones autenticadas sean inmediatas.

Datos: Los datos de la aplicación provienen de un proceso de Web Scraping automatizado. Se utiliza Playwright para navegar por las webs de tiendas de mascotas reales y extraer información actualizada (nombre, precio, fotos). Estos datos se limpian y se guardan en PostgreSQL para que la app Android los consuma mediante la API.

Arquitectura del Backend:

Node.js con Express y la base de datos es PostgreSQL. Se eligió por ser una base de datos relacional potente, ideal para manejar el catálogo de productos y usuarios con total integridad. Además, el uso del ORM (Prisma). Se utiliza Prisma como puente entre el código y la base de datos. Es interesante usar Prisma porque se supone ofrece un tipado fuerte, lo que evita errores al realizar consultas y facilita mucho el mantenimiento del código frente a usar SQL puro, aunque también se podría haber usado Selenium

11. Tests y Validaciones

Gestión de Errores y Estados: He creado un par pantallas específicas para cuando no hay internet, cuando la búsqueda no da resultados o mientras la app está cargando. Así el usuario siempre sabe qué está pasando y no se encuentra con una pantalla vacía. Similar al mensaje en la app del reloj de no hay servidor. Viene bien por si se cae el servidor, además es solo ver si hay algún error y no carga, entonces mostrar esos mensajes.

Rendimiento y Fluidez: Utilizo listas inteligentes (LazyLists) que solo cargan los productos que se están viendo en ese momento, ahorrando batería y memoria que por lo visto es lo ideal para este tipo de e-commers. Además, la librería Coil se encarga de descargar y guardar las imágenes de forma eficiente para que el scroll sea totalmente fluido.

11.1. Capturas y Evidencias

1. Catálogo Principal

Es la pantalla de entrada. Aquí se ve la cuadrícula adaptable con los productos scrapeados, los precios y el botón de "añadir con huella".



2. Inicio de Sesión

Pantalla limpia de autenticación. Permite al usuario identificarse para que su carrito y perfil sean persistentes.



3. Registro de Nuevo Usuario

Formulario completo que incluye el selector de fecha de nacimiento.

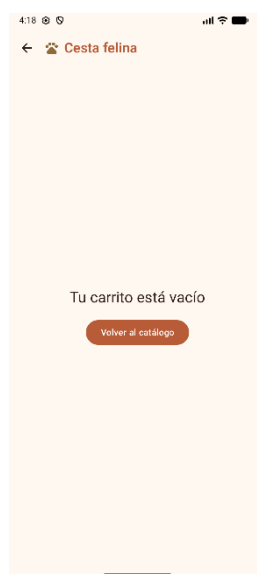
4. Detalles de Producto

Al pulsar en un producto, se abre esta vista con la imagen en alta resolución (gestionada por Coil) y la descripción técnica.



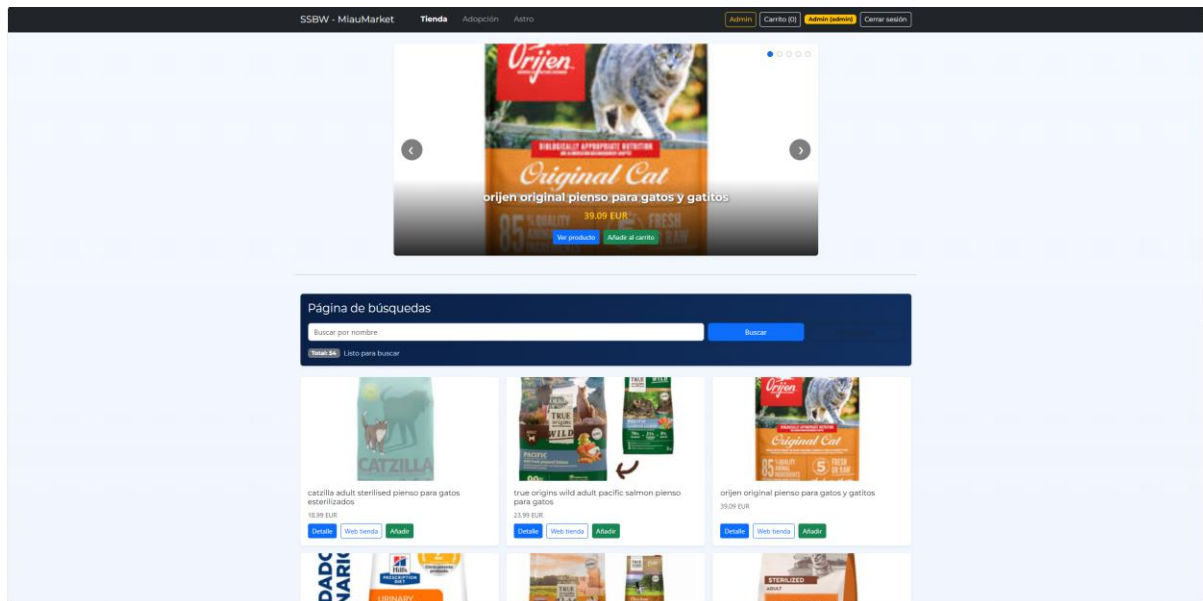
5. Cesta Felina / Carrito

Gestión local de los productos. Aquí se aplica la lógica de negocio para sumar totales y gestionar las cantidades.



6. API/Backend

Web subida con la lógica de negocio y back junto a la base de datos subida.



7. DigitalOcean

Gestión de back en la nube con datos y algunos componentes desde el panel de gestión.

Back to Apps

miaumarket

Add components
Actions

Healthy
first-project
FRA1
https://miaumarket-fw87p.ondigitaloce...

aaronblanco's [deployment](#) went live at 06:14:33 PM

Live App

Overview

Insights

Activity

Runtime Logs

Console

Networking

Settings

Components:

miaumarket App

ssbw Web Service

dev-db-436213 Database

Component Settings: ssbw

Info

ssbw
Web Service

Resource Size

\$24.00/mo
1 GB RAM | 1 Shared vCPU x 2 | 150 GB bandwidth
2 Containers

Edit

Source

Repository: <https://github.com/aaronblanco/SSBW>

Branch: main

Source Directory: /

Hash: ba57313

Autodeploy: On

Edit

12. Enlaces y Recursos

- **Código del Backend & Scraper:** [GitHub - SSBW](#)
- **Web Backend/Api:** [SSBW - Inicio](#)
- **Documentación de Diseño:** [Navegabilidad \(Draw.io\)](#)
- **Librerías Core Utilizadas:** Hilt, Retrofit, Coil, DataStore, Navigation Compose.